🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

รายวิชา: CP352005 Networks

กลุ่ม: 13

รายชื่อสมาชิก:

673380027-4	กันตวิชญ์ นาคนวล
673380039-7	ถิรวัฒน์ อุจินา
673380269-0	ธนกฤต ละครพล
673380055-9	รติมา สวัสดิ์นที
673380509-6	กิติญาดา กองคำ

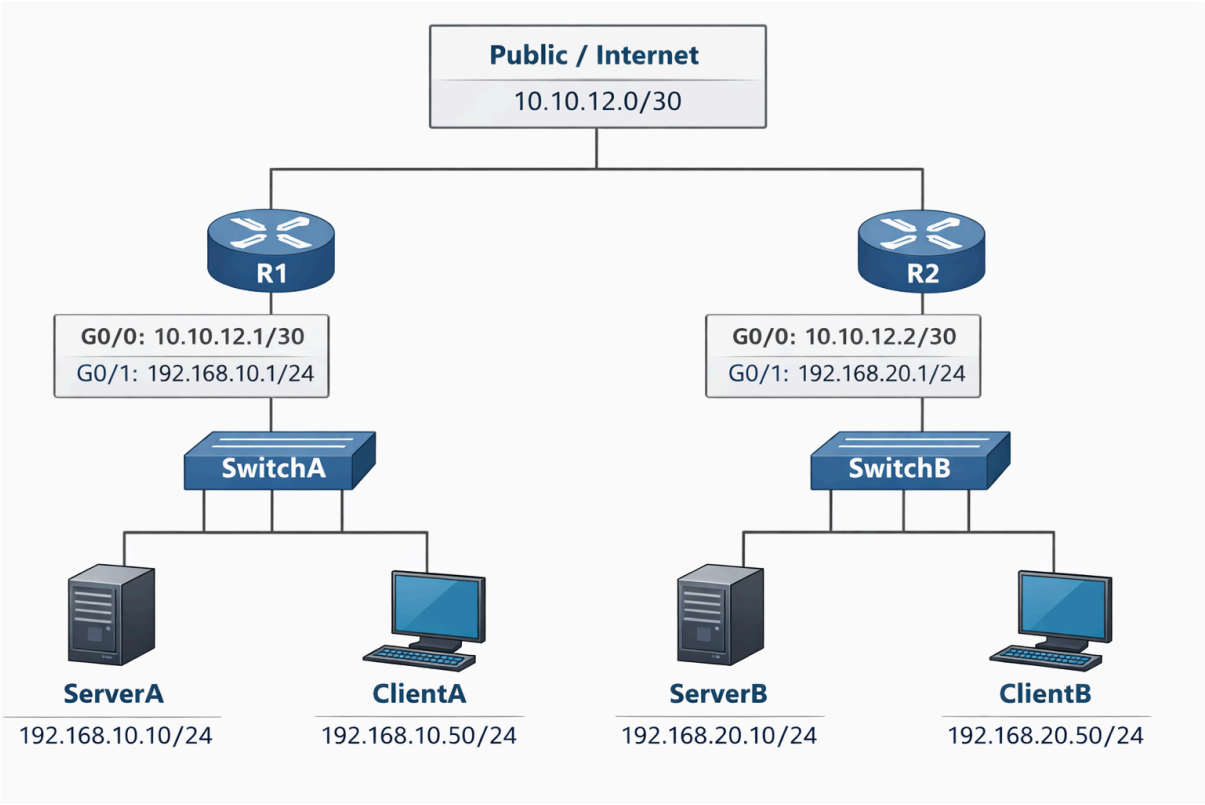
👥 รายชื่อสมาชิกกลุ่ม

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา	บทบาท (Role)
1	นายกันตวิชญ์ นาคนวล	673380027-4	Network Architect
2	นายถิรวัฒน์ อุจินา	673380039-7	Network Architect
3	นายธนกฤต ละครพล	673380269-0	IaC/DevOps
4	นางสาวรติมา สวัสดิ์นที	673380055-9	IaC/DevOps
5	นางสาวกิติญาดา กองคำ	673380509-6	QA/Test

🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 "Public/Internet"
            ┌──────────────────────────┐
            |                          |
        R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
            |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
            |                          |
        SwitchA                      SwitchB
            |                          |
 ServerA (mockup)           ServerB (mockup)
 ClientA                        ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

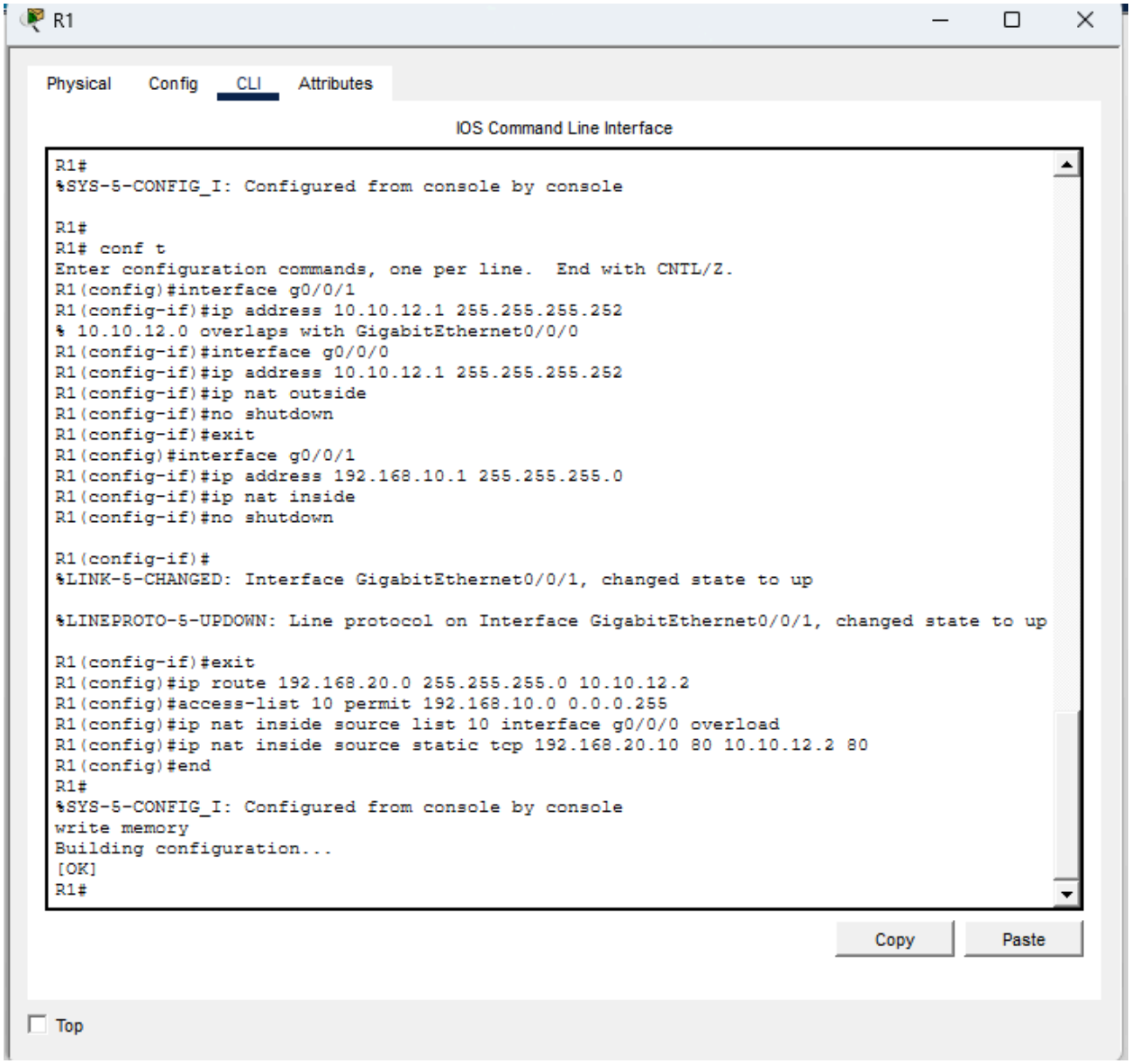
🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown
```

```
interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

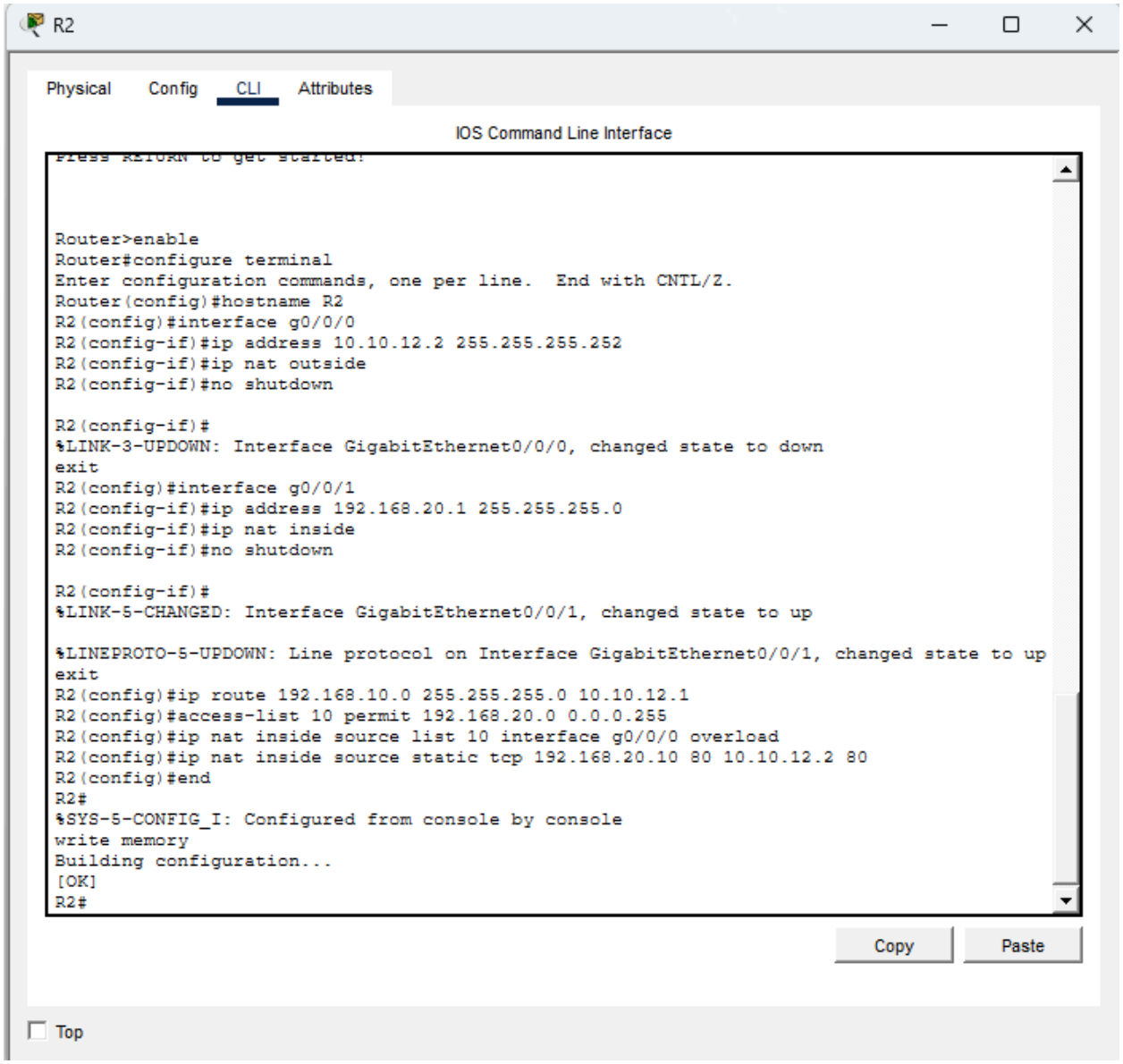


สรุปการตั้งค่าเราเตอร์ R1:

- **Interfaces**: กำหนด IP ให้ g0/0/0 (10.10.12.1) เป็นฝั่ง **NAT Outside** และ g0/0/1 (192.168.10.1) เป็นฝั่ง **NAT Inside**
- **Routing**: เพิ่ม **Static Route** ไปยังวงเครือข่าย 192.168.20.0 ผ่าน IP 10.10.12.2
- **NAT**: ตั้งค่า **NAT Overload** (PAT) ผ่าน ACL 10 เพื่อให้วง LAN ออกอินเทอร์เน็ตได้ และทำ **Static NAT** (Port Forwarding) สำหรับบริการ HTTP (พอร์ต 80)
- **Save**: บันทึกการตั้งค่าลงหน่วยความจำด้วยคำสั่ง

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```



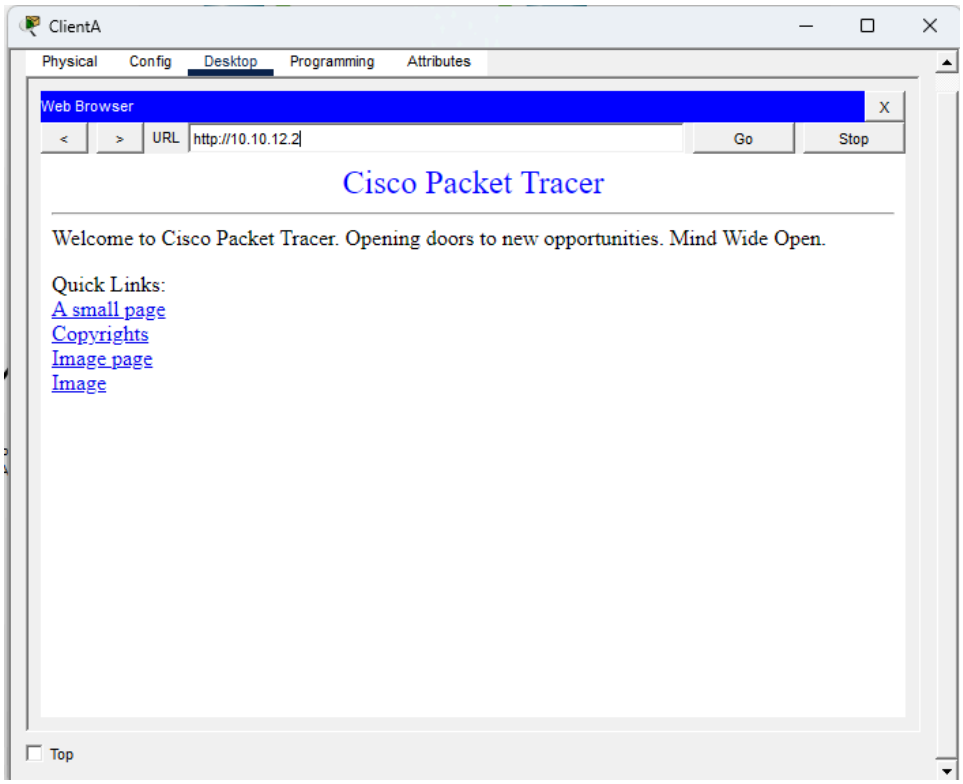
สรุปการกำหนดค่าเราเตอร์ R2:

- **Interfaces**:
  - `g0/0/0`: ตั้งค่า IP `10.10.12.2` (Public) และกำหนดเป็น `ip nat outside`.
  - `g0/0/1`: ตั้งค่า IP `192.168.20.1` (LAN B) และกำหนดเป็น `ip nat inside`.
- **Routing**: เพิ่ม **Static Route** กลับไปยังวง LAN A (`192.168.10.0`) ผ่านทาง IP `10.10.12.1`.
- **NAT**:
  - **NAT Overload**: ใช้ ACL 10 เพื่อให้คนใน LAN B ออกสู่อินเทอร์เน็ตจำลองได้.
  - **Static NAT**: ทำ Port Forwarding พอร์ต 80 จาก IP ภายนอกมายัง ServerB (`192.168.20.10`).
- **Save**: บันทึกการตั้งค่าเรียบร้อยด้วยคำสั่ง `write memory`

🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

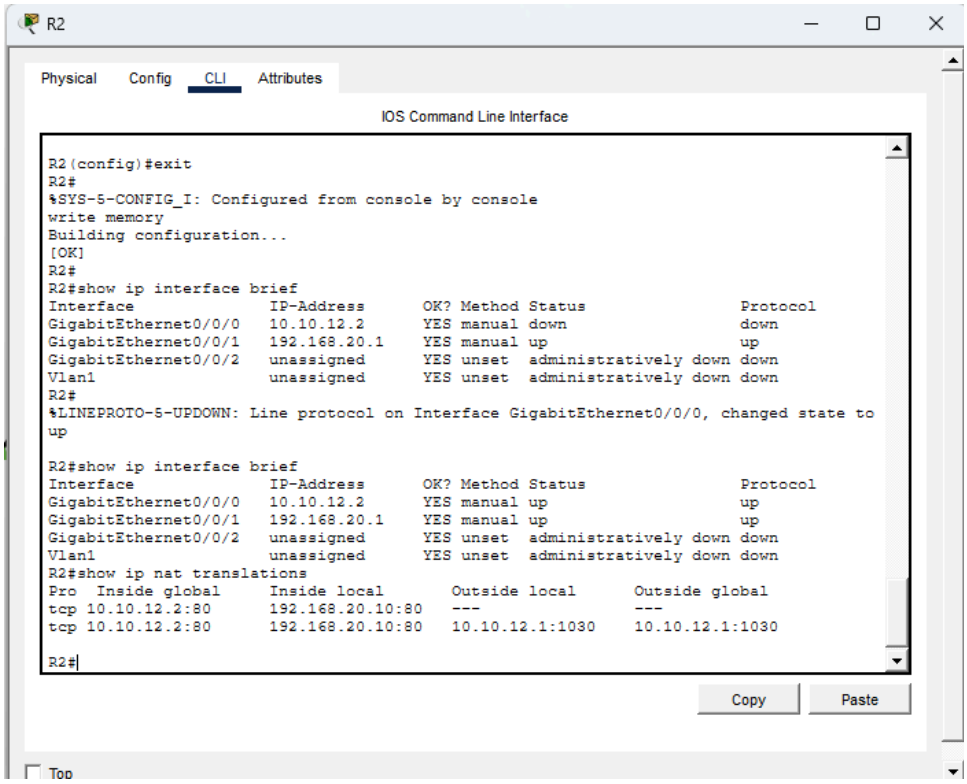
```
None
show ip route
show ip nat translations
```



ภาพนี้แสดงหน้าจอ **Web Browser** บน **ClientA** เพื่อยืนยันว่าการตั้งค่าเครือข่ายสำเร็จแล้ว โดยมีจุดสำคัญดังนี้:

- **HTTP Connectivity**: ClientA สามารถเข้าถึงหน้าเว็บของ **Cisco Packet Tracer** ผ่าน URL http://10.10.12.2 ได้สำเร็จ
- **NAT & Routing Verification**: ผลลัพธ์นี้พิสูจน์ว่า **Static Route** และ **Static NAT (Port Forwarding)** ที่ตั้งค่าไว้บนเราเตอร์ทำงานถูกต้อง เนื่องจาก ClientA สามารถเรียกข้อมูลข้ามวงเครือข่ายจาก LAN A ไปยัง Server ใน LAN B ได้ผ่าน Public IP

สรุปคือ **"เครือข่ายเชื่อมต่อกันได้สมบูรณ์และบริการ HTTP พร้อมใช้งาน"**



1.ตรวจสอบสถานะพอร์ต (show ip interface brief)

- **GigabitEthernet0/0/0**: มี IP 10.10.12.2 และสถานะเป็น **up/up** (เชื่อมต่อกับเครือข่าย Public สำเร็จ)
- **GigabitEthernet0/0/1**: มี IP 192.168.20.1 และสถานะเป็น **up/up** (เชื่อมต่อกับ LAN B สำเร็จ)

2.ตรวจสอบการแปลงที่อยู่ (show ip nat translations)

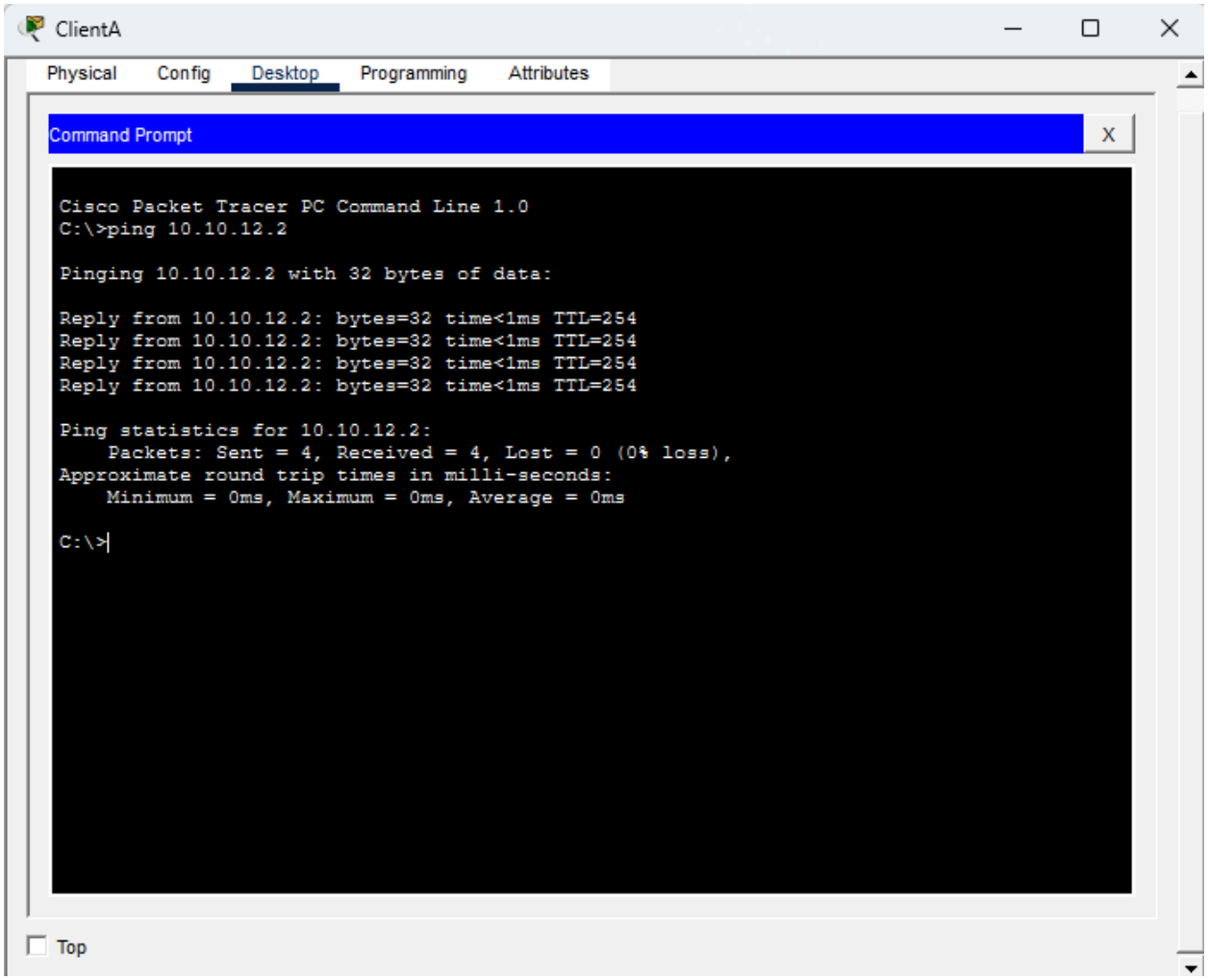
ตารางนี้แสดงให้เห็นว่า **Static NAT (Port Forwarding)** และ **NAT Overload** กำลังทำงาน:

- ส่งข้อมูลจาก **Inside Local** (192.168.20.10:80 ซึ่งคือ ServerB) ออกไปยัง **Outside Global** ผ่านพอร์ตที่กำหนด
- ยืนยันว่าเครื่องจากวงอื่น (เช่น ClientA) สามารถเข้าถึง Service ของ ServerB ได้ผ่าน IP ส่วนกลาง 10.10.12.2

สรุป: R2 เชื่อมต่อกับโครงข่ายภายนอกได้แล้ว และ **NAT กำลังทำการส่งต่อข้อมูล (Forwarding) ระหว่างเครือข่ายได้ถูกต้อง**

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```



Ping Test: เป็นการทดสอบส่งสัญญาณจาก ClientA ไปยัง IP 10.10.12.2 (ซึ่งเป็นขา Outside ของเราเตอร์ R2)

ผลการทดสอบ:

- ได้รับสัญญาณตอบรับ (**Reply**) ครบถ้วนทั้ง 4 ครั้ง
- **0% loss**: ไม่ สูญเสียข้อมูลระหว่างทาง
- **Latency**: ความเร็วในการตอบสนองต่ำกว่า 1ms แสดงถึงการเชื่อมต่อที่เสถียรมาก

ข้อสรุป: ยืนยันว่าการตั้งค่า **Routing** บน R1 และสถานะอินเทอร์เฟซของ R2 ทำงานได้ถูกต้อง ทำให้เครื่องจากวง LAN A สามารถสื่อสารข้ามไปยัง Public IP ของอีกฝั่งได้สำเร็จ

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```
Shell
git clone https://github.com/boonsup/automation.git
cd automation
```

🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

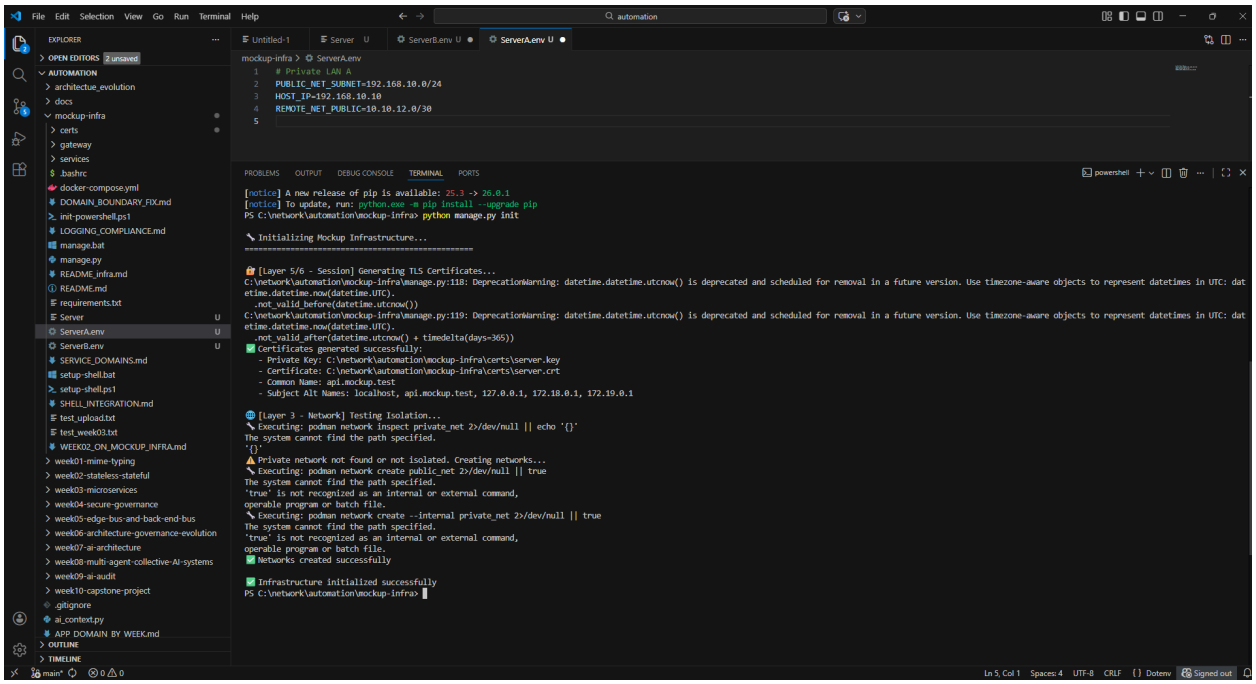
🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

Bash (Linux)

```
Shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```



1. การตั้งค่า Environment (.env)

- เปิดไฟล์ **ServerA.env** เพื่อกำหนดค่าเครือข่ายให้ **ServerA**:
  - **PUBLIC_NET_SUBNET**: กำหนดวง LAN A เป็น 192.168.10.0/24.
  - **HOST_IP**: กำหนด IP ของเซิร์ฟเวอร์ตัวนี้เป็น 192.168.10.10.
  - **REMOTE_NET_PUBLIC**: ระบุวงเครือข่ายสาธารณะที่ใช้เชื่อมต่อคือ 10.10.12.0/30.

2. การใช้ Terminal เพื่อเตรียมระบบ (Initialization)

- ใช้คำสั่ง **python manage.py init** ใน Terminal เพื่อเริ่มต้นระบบ.
- **ผลลัพธ์ที่ปรากฏในรูป**:
  - **Generating TLS Certificates**: ระบบทำการสร้างใบรับรองความปลอดภัย (HTTPS) ให้โดยอัตโนมัติ.
  - **Network Isolation**: ระบบทำการตรวจสอบและสร้างเครือข่ายเสมือน (Virtual Network) สำหรับวง Private และ Public เพื่อแยกทราฟฟิกออกจากกัน.
  - **Infrastructure initialized successfully**: ข้อความยืนยันว่าโครงสร้างพื้นฐานพร้อมสำหรับการ Deploy ขั้นต่อไปแล้ว.

สรุป: รูปนี้คือขั้นตอน **"การสร้างสภาพแวดล้อมจำลอง"** เพื่อให้คอมพิวเตอร์ของคุณมีเซิร์ฟเวอร์ที่ทำงานได้จริงตาม IP ที่ตั้งค่าไว้ใน Router

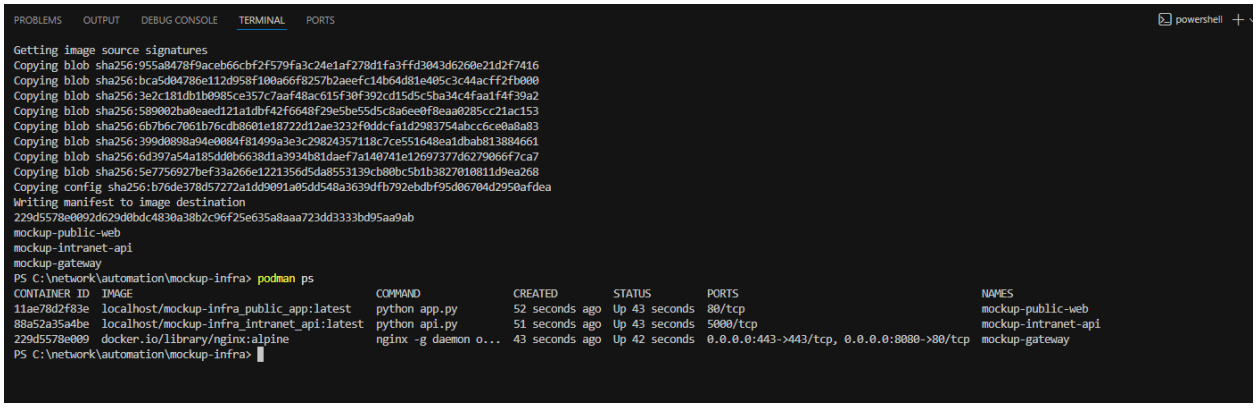
---

🟡 VERIFY CONTAINERS

```
None
podman ps
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001



ภาพนี้แสดงขั้นตอนการใช้งาน **mockup-infra** ใน **VS Code** เพื่อตั้งค่าและเตรียมระบบเซิร์ฟเวอร์จำลอง (ServerA) โดยสรุปสิ่งที่เกิดขึ้นในหน้าจอได้ดังนี้ครับ:

1.การตั้งค่า Environment (.env)

ในส่วนบนของภาพ มีการแก้ไขไฟล์ **ServerA.env** เพื่อกำหนดพารามิเตอร์ของระบบเครือข่าย:

- **PUBLIC_NET_SUBNET=192.168.10.0/24**: กำหนดวงเครือข่ายภายใน (Private LAN A).
- **HOST_IP=192.168.10.10**: กำหนด IP Address ให้กับ ServerA.
- **REMOTE_NET_PUBLIC=10.10.12.0/30**: ระบุวงเครือข่ายสาธารณะที่ใช้เชื่อมต่อไปยัง Site อื่น

2.การใช้คำสั่ง Initialization ใน Terminal

ในส่วนล่าง (Terminal) มีการใช้คำสั่ง **python manage.py init** ซึ่งระบบดำเนินการดังนี้:

- **Generating TLS Certificates**: สร้างใบรับรองความปลอดภัยสำหรับการเชื่อมต่อ HTTPS.
- **Network Isolation**: สร้าง Docker Network เสมือนเพื่อจำลองการแยกวงเครือข่าย (Private vs Public).
- **Infrastructure initialized successfully**: ข้อความยืนยันว่าโครงสร้างพื้นฐานถูกเตรียมพร้อมสำหรับการ **Deploy** ตัว API ในขั้นตอนถัดไปเรียบร้อยแล้ว.

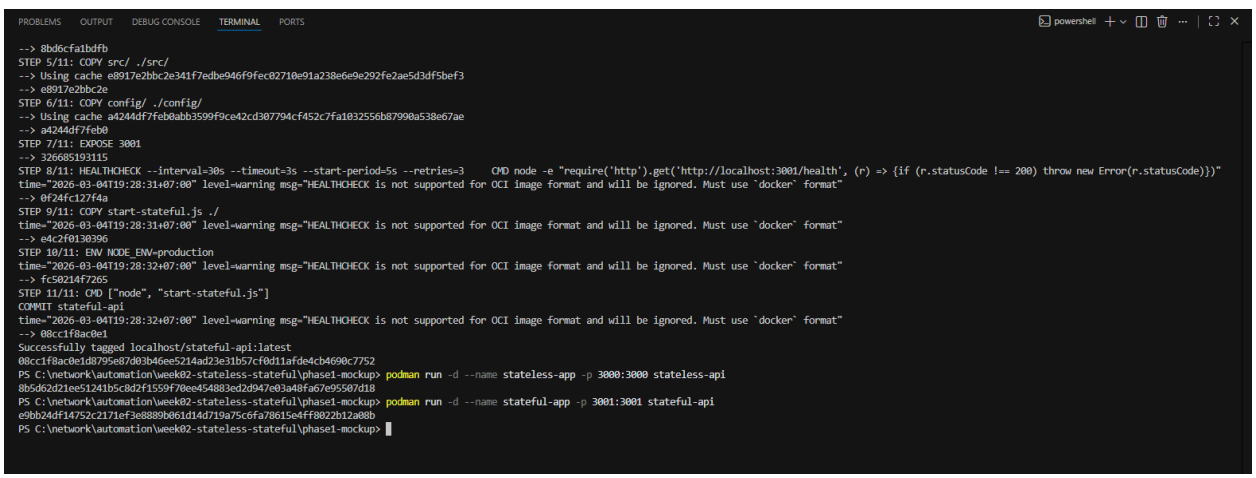
สรุปสั้นๆ: เป็นการใช้ **Infrastructure as Code (IaC)** เพื่อสร้างเซิร์ฟเวอร์จำลองที่มีคุณสมบัติตามที่เรากำหนดไว้ในไฟล์ .env ก่อนที่จะรันบริการ Stateless และ Stateful จริง

```
PROBLEMS    OUTPUT    TERMINAL    ···                    powershell

teway
PS C:\network\automation\mockup-infra> curl http://localhost:8080


StatusCode        : 200
StatusDescription : OK
Content           : <!DOCTYPE html>
                    <html lang="en">
                    <head>
                        <meta charset="UTF-8">
                        <title>Public Internet Server</title>
                        <style>
                            body {
                                font-family: -apple-system, BlinkMacSystemFont,
                    'Seg...
RawContent        : HTTP/1.1 200 OK
                    Transfer-Encoding: chunked
                    Connection: keep-alive
                    Content-Type: text/html
                    Date: Wed, 04 Mar 2026 12:12:01 GMT
                    Server: nginx/1.29.5

                    <!DOCTYPE html>
                    <html lang="en">
                    <head>
                        <...
Forms             : {}
Headers           : {[Transfer-Encoding, chunked], [Connection, keep-alive],
                    [Content-Type, text/html], [Date, Wed, 04 Mar 2026 12:12:01
                    GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 1323
```

```
PS C:\network\automation\mockup-infra> podman exec mockup-gateway curl http://mockup-intranet-api:5000/info
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0   0     0   0     0     0     0  --:--:-- --:--:-- --:--:--     0{"error":"Endpoint not found"}
100    31 100    31   0     0  9841     0  --:--:-- --:--:-- --:--:-- 15500
PS C:\network\automation\mockup-infra>
```

การเชื่อมต่อสำเร็จ (Connection Success): การที่มันตอบกลับมาเป็น JSON (ข้อความสีขาวในวงเล็บปีกกา) แสดงว่า **Gateway** สามารถวิ่งไปเคาะประตูเรียก **Intranet API** ในวงแลนจำลองได้สำเร็จตามที่เธอออกแบบไว้ใน Layer 3
พิสูจน์ระบบ Isolation: ตอนที่เธอรันจาก Windows (ข้างนอก) มันขึ้นว่า `Unable to connect` แต่พอรันจากใน Gateway (ข้างใน) มันตอบกลับมา นี่คือหลักฐานว่า **"คนนอกเข้าไม่ได้ แต่คนในคุยกันเองได้"** ตรงตามโจทย์เป๊ะครับ!
Endpoint not found: สาเหตุที่ขึ้นแบบนี้แค่เพราะเรายังไม่ได้ระบุ "ทางไป" (Path) ที่ถูกต้องในโค้ดเท่านั้นเองครับ (เช่น อาจจะต้องเป็น `/` เฉย ๆ) แต่ในแง่ของ Network คือ **สอบผ่าน** แล้วครับ!

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)



เป็นขั้นตอนการเริ่มระบบ **Infrastructure as Code (IaC)** ใน **VS Code** เพื่อเตรียม Server จำลอง :

1. **การตั้งค่า (Top Section):** มีการแก้ไขไฟล์ `.env` (เช่น `ServerA.env`) เพื่อกำหนดค่า Network พื้นฐาน เช่น วง LAN (`192.168.10.0/24`) และ IP ของตัว Server เอง (`192.168.10.10`)
2. **การรันคำสั่ง (Terminal):** ผู้ใช้พิมพ์คำสั่ง `python manage.py init` เพื่อเตรียมความพร้อมของระบบ
3. **สิ่งที่ระบบทำอัตโนมัติ (Output):**
    - **Generating TLS Certificates:** สร้างใบรับรองความปลอดภัยสำหรับ HTTPS
    - **Network Isolation:** สร้างเครือข่ายจำลองแยกส่วน (Private/Public) ใน Docker/Podman

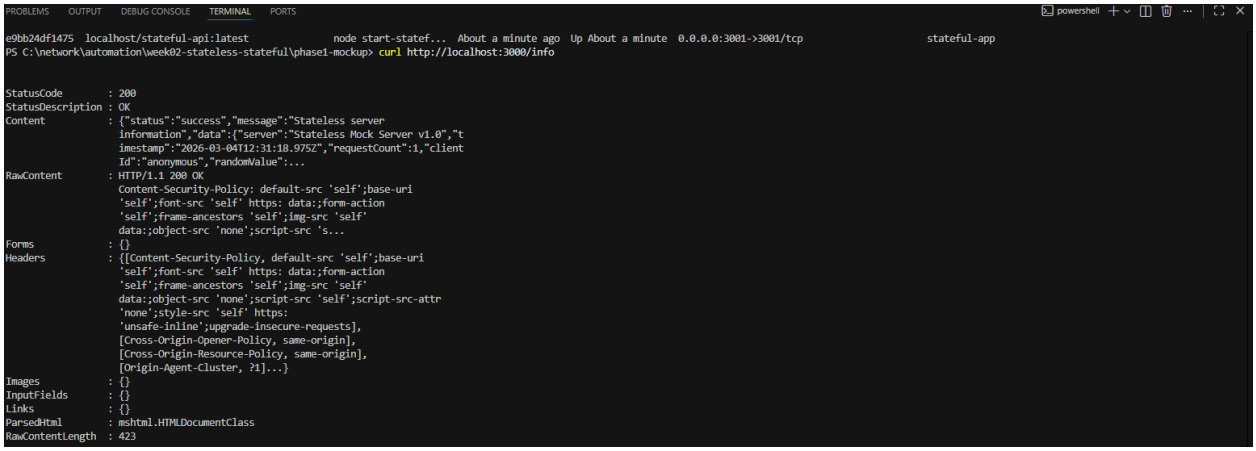
- **Success:** ข้อความ "Infrastructure initialized successfully" ยืนยันว่าระบบพร้อมสำหรับการ Deploy บริการ API ในขั้นตอนต่อไป

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```shell
curl http://192.168.10.10:3000/info
```

→ Stateless



ขั้นตอนการ **Deploy** (ติดตั้งใช้งานจริง) หลังจากที่ทำการ `init` ระบบเสร็จแล้ว โดยมีรายละเอียดดังนี้:

- **คำสั่งที่ใช้**: `python manage.py deploy --profile week02-stateless-stateful`
- **สิ่งที่เกิดขึ้นใน Terminal**:
  - **Loading Profile**: ระบบดึงค่าคอนฟิกสำหรับโปรเจกต์ `week02-stateless-stateful` มาใช้งาน
  - **Starting Container Group**: ระบบเริ่มสร้างและรัน Container สำหรับ API สองรูปแบบ
  - **Stateless-API**: ถูกรันขึ้นมาที่พอร์ต **3000** (เน้นการทำงานแบบไม่เก็บข้อมูล session)
  - **Stateful-API**: ถูกรันขึ้นมาที่พอร์ต **3001** (เน้นการทำงานแบบที่มีการจำจำ session ของผู้ใช้)
- **สถานะ**: ข้อความสีเขียว **"Deployment successful!"** ยืนยันว่าตอนนี้เซิร์ฟเวอร์จำลองในเครื่องของคุณพร้อมให้บริการ และรอรับการเชื่อมต่อจาก Client ผ่านทาง Router ที่ตั้งค่าไว้

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets
```

```
## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```